# Deep Learning Notes

Exam Oriented Preparation Notes

Based on University Syllabus and PYQs

# Contents


## Unit 1: From Machine Learning to Deep Learning 3

**1 Math Behind Machine Learning 3**
- 1.1 Statistics and Probability Fundamentals . . . 3
- 1.2 Probability Distributions . . . 4
- 1.3 Samples vs. Population, Resampling, Selection Bias . . . 5
- 1.4 Likelihood and Maximum Likelihood Estimation (MLE) . . . 6

**2 Bayesian Statistics, Supervised and Unsupervised Learning 6**
- 2.1 Bayesian Statistics . . . 6
- 2.2 Supervised vs. Unsupervised Learning . . . 7

**3 Stochastic Gradient Descent (SGD) 8**

**4 Building a Machine Learning Algorithm 9**

**5 Challenges Motivating Deep Learning 10**

## Unit 2: Fundamentals of Deep Networks 12

**6 Common Architectural Principles of Deep Networks 12**

**7 Deep Feedforward Networks – The XOR Example 13**

**8 Gradient-Based Learning 15**
- 8.1 Forward Propagation . . . 15
- 8.2 Backpropagation Algorithm . . . 16

**9 Hidden Units and Activation Functions 17**
- 9.1 Architecture Design . . . 18

## Unit 3: Regularization and Optimization for Training Deep Models 20

**10 Parameter Norm Penalties 20**
- 10.1 Norm Penalties as Constrained Optimization . . . 21
- 10.2 Dataset Augmentation and Noise Robustness . . . 21

**11 Semi-Supervised Learning, Multi-Task Learning, Early Stopping, Dropout, and Ensemble Methods 22**
- 11.1 Semi-Supervised Learning . . . 22

11.2 Multi-Task Learning (MTL) . . . . . 22
11.3 Early Stopping . . . . . 23
11.4 Dropout . . . . . 24
11.5 Bagging and Ensemble Methods . . . . . 26
11.6 Adversarial Training . . . . . 27

**12 Challenges in Neural Network Optimization** **27**

**13 Basic Optimization Algorithms** **28**
13.1 Momentum . . . . . 28

**14 Algorithms with Adaptive Learning Rates** **28**

**15 Parameter Initialization Strategies** **30**

**16 Approximate Second-Order Methods and Batch Normalization** **31**
16.1 Batch Normalization . . . . . 31

# Unit 1: From Machine Learning to Deep Learning

## 1 Math Behind Machine Learning

### 1.1 Statistics and Probability Fundamentals

- **Random Variable**: A variable whose value is determined by the outcome of a random experiment. Can be **discrete** (countable values) or **continuous** (uncountable real values).
- **Probability**: A measure of how likely an event is to occur, ranging from 0 (impossible) to 1 (certain).
- **Joint Probability** $P(A \cap B)$: Probability that both events $A$ and $B$ occur simultaneously. For independent events: $P(A \cap B) = P(A) \cdot P(B)$.
- **Marginal Probability** $P(A)$: Probability of an event $A$ regardless of the outcome of other events. Obtained by summing/integrating the joint probability over all values of the other variable.
- **Conditional Probability** $P(A \mid B)$: Probability of event $A$ given that event $B$ has already occurred. It updates the likelihood of $A$ based on new evidence $B$.

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$

- **Prior Probability** $P(H)$: The initial probability of a hypothesis $H$ before observing any data. Encodes prior belief or domain knowledge about how likely $H$ is.
- **Likelihood** $P(D \mid H)$: How probable the observed data $D$ is if hypothesis $H$ is true. Measures model fitness to data.
- **Posterior Probability** $P(H \mid D)$: The updated probability of $H$ after observing data $D$. Combines prior belief with evidence from data.

**Bayes Theorem** (derivation from conditional probability definition):

$$\boxed{P(H \mid D) = \frac{P(D \mid H) \cdot P(H)}{P(D)}}$$

$$\textit{Posterior} \propto \textit{Likelihood} \times \textit{Prior}$$

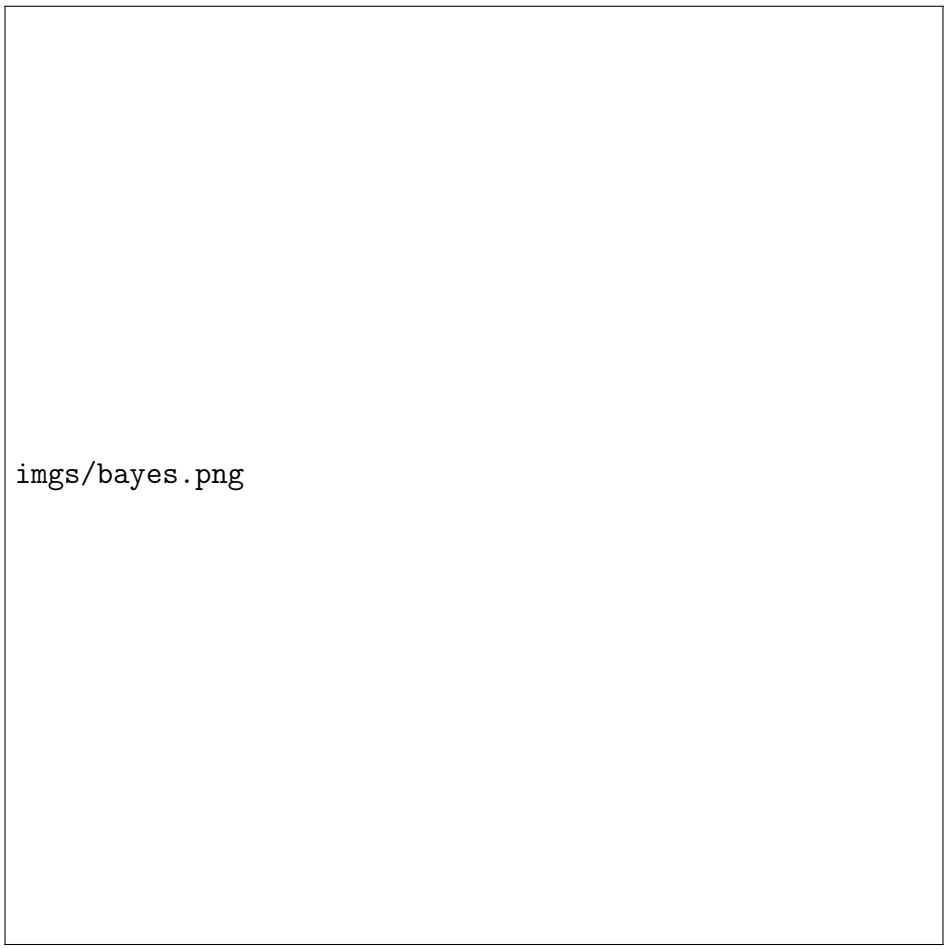



Figure 1: Bayes Theorem: updating prior belief with evidence to obtain posterior

## 1.2 Probability Distributions

- **Bernoulli Distribution**: Models a single binary trial. Takes value 1 with probability $p$ and 0 with probability $1-p$. Used in binary classification output neurons.

$$P(X = x) = p^x(1-p)^{1-x}, \quad x \in \{0, 1\}$$

- **Binomial Distribution**: Number of successes in $n$ independent Bernoulli trials.

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

- **Gaussian (Normal) Distribution**: Continuous, symmetric, bell-shaped distribution defined by mean $\mu$ and variance $\sigma^2$. Most weight initialization methods

assume Gaussian noise.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

- **Categorical (Multinoulli) Distribution**: Generalization of Bernoulli to $k$ classes. Each class $i$ has probability $p_i$ where $\sum_i p_i = 1$. Used in multi-class classification (softmax output).
- **Uniform Distribution**: All outcomes in $[a, b]$ equally likely. Used in weight initialization to break symmetry.

$$f(x) = \frac{1}{b-a}, \quad a \leq x \leq b$$

## 1.3 Samples vs. Population, Resampling, Selection Bias

- **Population**: The complete set of all possible data points of interest. In practice, true population is inaccessible; we rely on samples.
- **Sample**: A finite subset drawn from the population used to estimate population properties. Model must **generalize** from sample to population.
- **i.i.d. Assumption**: Training samples are assumed to be **independently and identically distributed** (drawn from the same distribution, independently of each other).
- **Bootstrap Resampling**: Sampling **with replacement** from the training set to create multiple datasets. Used to estimate the variance of a model's performance.
- **Cross-Validation**: Partitions data into $k$ folds; trains on $k-1$ and validates on 1, rotating $k$ times. Reduces variance in performance estimates.
- **Selection Bias**: Occurs when the collected sample is **not representative** of the true population. Example: training a face detector only on fair-skinned subjects leads to poor performance on dark-skinned subjects. Leads to biased and poorly generalizing models.

### 1.4 Likelihood and Maximum Likelihood Estimation (MLE)

- **Likelihood Function** $\mathcal{L}(\theta; X)$: A function of parameters $\theta$ that measures how probable the observed data $X$ is under those parameters. Note: likelihood is *not* a probability over $\theta$.
- **Maximum Likelihood Estimation (MLE)**: Finds the parameter values $\hat{\theta}$ that maximize the probability of observing the data.

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} \; P(X \mid \theta) = \arg\max_{\theta} \prod_{i=1}^{N} P(x_i \mid \theta)$$

- **Log-Likelihood**: Taking the log converts the product to a sum (easier to optimize; log is monotone so maximizer is unchanged):

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta} \sum_{i=1}^{N} \log P(x_i \mid \theta)$$

- **Cross-Entropy Loss**: Minimizing cross-entropy is equivalent to maximizing the log-likelihood under a categorical distribution.
- **Limitation of MLE**: Can overfit when data is scarce (no regularization). Remedied by MAP.

## 2 Bayesian Statistics, Supervised and Unsupervised Learning

### 2.1 Bayesian Statistics

- **Frequentist View**: Parameters $\theta$ are fixed but unknown constants. Probability = long-run frequency of events. MLE is a frequentist method.
- **Bayesian View**: Parameters $\theta$ are **random variables** with a probability distribution. Probability = degree of belief. Updated as data arrives.
- **Posterior Predictive**: Full Bayesian inference integrates over all possible $\theta$ values:

$$P(y^* \mid x^*, X) = \int P(y^* \mid x^*, \theta)\, P(\theta \mid X)\, d\theta$$

- **MAP (Maximum A Posteriori) Estimation**: Instead of integrating, find the single most probable parameter given data and prior:

$$\hat{\theta}_{\text{MAP}} = \arg\max_{\theta} \left[\log P(D \mid \theta) + \log P(\theta)\right]$$

- **Regularization Connection**: Gaussian prior on $\theta$ gives MAP equivalent to **L2 regularization**; Laplacian prior gives MAP equivalent to **L1 regularization**.

| **Aspect** | **Frequentist (MLE)** | **Bayesian (MAP / Full Bayes)** |
|---|---|---|
| **Parameters** | Fixed unknowns | Random variables with distribution |
| **Probability** | Long-run frequency | Degree of belief |
| **Overfitting** | Prone (no prior) | Regularized by prior |
| **Computation** | Fast | Often slow (integration) |

## 2.2 Supervised vs. Unsupervised Learning

| **Aspect** | **Supervised Learning** | **Unsupervised Learning** |
|---|---|---|
| **Data** | Labeled pairs $(x_i, y_i)$ | Unlabeled inputs $x_i$ only |
| **Goal** | Learn mapping $x \to y$ | Discover hidden structure |
| **Output** | Class label / real value | Cluster, embedding, density |
| **Examples** | Classification, Regression | Clustering, PCA, Autoencoders |
| **Algorithms** | SVM, DNN, Decision Tree | K-Means, VAE, GAN |
| **Evaluation** | Accuracy, MSE, F1 | Silhouette, reconstruction error |
| **DL usage** | CNN (vision), RNN (NLP) | Autoencoders, GANs |

# 3 Stochastic Gradient Descent (SGD)

- **Gradient Descent**: An iterative first-order optimization algorithm that updates parameters in the direction of the **negative gradient** of the loss function to minimize it.
- **General Update Rule**:

$$\theta \leftarrow \theta - \alpha \cdot \nabla_\theta \mathcal{L}(\theta)$$

$$\text{where } \alpha = \text{learning rate}, \quad \nabla_\theta \mathcal{L} = \text{gradient of loss w.r.t. } \theta$$

- **Batch Gradient Descent**: Computes gradient using the **entire training dataset**. Gives exact gradient but is very slow and memory-intensive for large datasets.
- **Stochastic Gradient Descent (SGD)**: Uses **one randomly chosen sample** per update. Very fast updates, low memory, but high variance (noisy gradient) causing oscillatory convergence path.

$$\theta \leftarrow \theta - \alpha \cdot \nabla_\theta \mathcal{L}(\theta; x_i, y_i)$$

- **Mini-Batch Gradient Descent**: Uses a small **batch of $B$ samples** (typically 32 to 256). Best trade-off: more stable than pure SGD, faster than full batch. This is what is meant by "SGD" in most deep learning frameworks.

| Property | Batch GD | SGD | Mini-Batch GD |
|---|---|---|---|
| **Data per update** | All $N$ samples | 1 sample | $B$ samples |
| **Gradient quality** | Exact | Noisy | Approximate |
| **Speed per update** | Slowest | Fastest | Fast |
| **Memory use** | High | Low | Moderate |
| **Convergence** | Smooth | Oscillatory | Fairly smooth |
| **Parallelism** | Limited | None | GPU-friendly |

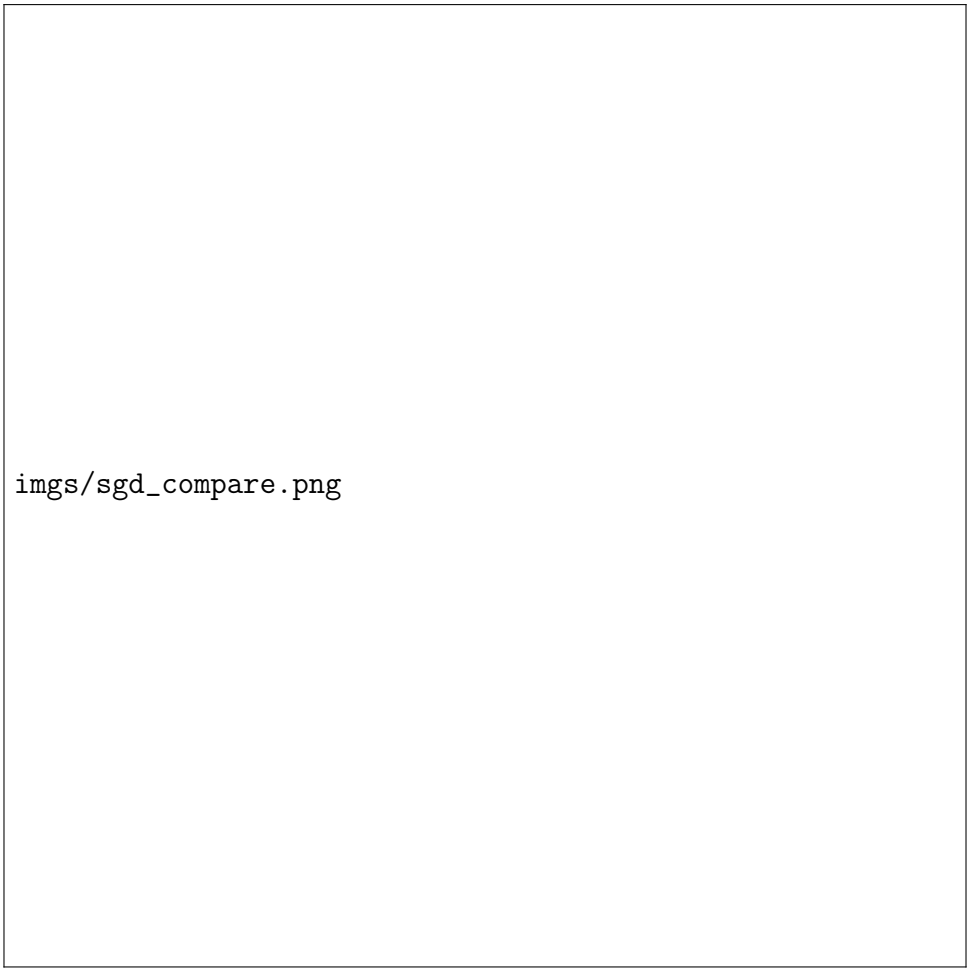



Figure 2: Loss curves: Batch GD (smooth), SGD (oscillatory), Mini-Batch GD (moderately smooth)

# 4 Building a Machine Learning Algorithm

1. **Dataset Collection and Preprocessing**: Collect representative data. Clean, normalize, and split into **train** / **validation** / **test** sets. Ensure the data is i.i.d. and free from selection bias.
2. **Model (Hypothesis Class)**: Define a parametric family of functions $f_\theta : \mathcal{X} \to \mathcal{Y}$ (e.g., linear model, neural network). The model's complexity determines what patterns it can represent.
3. **Loss / Cost Function**: Quantifies the discrepancy between model predictions

and true labels. Must be differentiable for gradient-based optimization.

$$\text{MSE: } \mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

$$\text{Cross-Entropy: } \mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)]$$

4. **Optimization Algorithm**: Minimize the loss by iteratively adjusting $\theta$ using gradient descent (SGD, Adam, etc.). Finds $\theta^* = \arg\min_\theta \mathcal{L}(\theta)$.
5. **Regularization**: Techniques (L1, L2, Dropout, Early Stopping) to prevent overfitting and improve generalization.
6. **Evaluation**: Measure performance on held-out **test data** using task-specific metrics: Accuracy, F1-score, AUC-ROC (classification); MSE, MAE (regression).
7. **Hyperparameter Tuning**: Search over learning rate, batch size, network depth/width, regularization strength using grid search, random search, or Bayesian optimization.

# 5 Challenges Motivating Deep Learning

- **Curse of Dimensionality**: As input dimension $d$ increases, the volume of the space grows exponentially. The number of training samples needed to adequately cover the space also grows exponentially, making traditional models (k-NN, kernel SVM) impractical for high-dimensional data such as images.
- **Feature Engineering Bottleneck**: Classical ML requires **handcrafted features** (HOG, SIFT, Bag-of-Words). This is domain-specific, time-consuming, and sub-optimal. Deep learning automatically learns **hierarchical representations** from raw data.
- **Local Smoothness Assumption**: Many classical models assume that nearby inputs produce similar outputs. This assumption fails for complex functions with many fine-grained variations (e.g., two visually similar images of different classes).
- **Limited Representational Capacity**: Shallow models (single-layer SVMs,

logistic regression) can only represent **linear or mildly non-linear** decision boundaries. Deep networks can approximate **exponentially more complex functions** with depth.

- **Scalability**: Classical ML often saturates in performance as data grows. Deep learning **scales** with data and compute (GPUs, TPUs), consistently improving with more data.
- **Deep Learning Solution**: Multiple stacked non-linear layers learn **distributed representations**: lower layers detect edges, middle layers detect shapes/textures, and higher layers detect semantic concepts (faces, objects). This hierarchical composition is extremely powerful.

# Unit 2: Fundamentals of Deep Networks

## 6 Common Architectural Principles of Deep Networks

- **Depth**: The number of layers in a network. Greater depth allows the network to represent more complex functions with fewer parameters by composing simple non-linear transformations. Each layer learns a progressively more abstract feature: edges → textures → parts → objects.
- **Width**: Number of units (neurons) per layer. Wider layers increase the representational capacity within a single transformation. There is a trade-off: depth is generally more parameter-efficient than width.
- **Feedforward (Acyclic) Architecture**: Information flows in one direction – from input to hidden layers to output – with no cycles. Also called **Multilayer Perceptron (MLP)**.
- **Parameter Sharing**: The same weights are reused across multiple positions or time steps. CNNs share filter weights across spatial positions; RNNs share weights across time steps. This **dramatically reduces** the number of parameters.
- **Modular / Compositional Design**: Networks are built from repeated standard blocks (convolutional blocks, LSTM cells, residual blocks). Allows easy scaling and component replacement.
- **Universal Approximation Theorem** (Cybenko, 1989; Hornik, 1991): A feedforward network with **at least one hidden layer** of sufficient width and a non-linear activation function can approximate **any continuous function** on a compact domain to arbitrary precision. Depth makes this much more parameter-efficient.
- **Non-linearity**: Without activation functions, stacking linear layers collapses to a single linear transformation ($W_n \cdots W_1 x = W_{\text{eff}} x$). Non-linear activations (ReLU, sigmoid) are essential for learning complex functions.

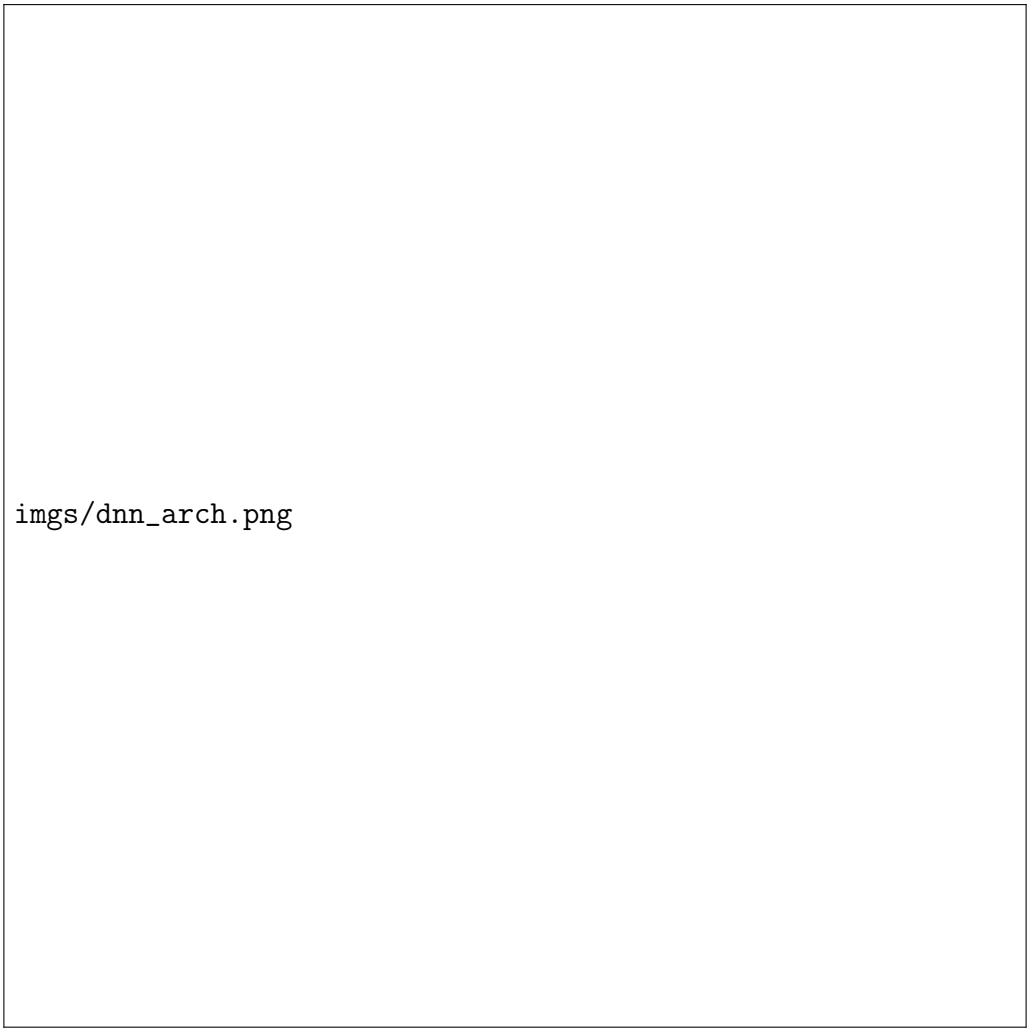



Figure 3: Deep Feedforward Network: Input layer → Hidden Layer 1 → Hidden Layer 2 → Output layer. Arrows represent weighted connections.

# 7 Deep Feedforward Networks – The XOR Example

**XOR Truth Table:**

| $x_1$ | $x_2$ | $x_1 \oplus x_2$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

- **Linear Inseparability**: No single straight line (hyperplane) in 2D input space

can correctly separate the four XOR points into two classes (0,0),(1,1) from (0,1),(1,0). A single-layer perceptron can only create linear decision boundaries.

- **Perceptron Output**: $\hat{y} = \text{step}(w_1 x_1 + w_2 x_2 + b)$. Since XOR is not linearly separable, no values of $w_1, w_2, b$ satisfy all four constraints simultaneously.
- **Solution – Hidden Layer**: Adding one hidden layer with non-linear activations transforms the input space into a new **learned feature space** where XOR *is* linearly separable.
- **Network with ReLU**: Inputs $x_1, x_2 \in \{0, 1\}$; one hidden layer with 2 ReLU units; output layer with 1 unit.

**One valid weight assignment:**

$$h_1 = \text{ReLU}(x_1 + x_2 - 0.5), \quad h_2 = \text{ReLU}(-x_1 - x_2 + 1.5)$$

$$y = h_1 - 2h_2 + 1 \quad \Rightarrow \quad \text{correctly computes XOR for all inputs}$$

- **Key Insight**: The hidden layer learns an **internal representation**. In the new space $(h_1, h_2)$, the XOR outputs become linearly separable. This illustrates why depth matters: each layer adds a level of abstraction.

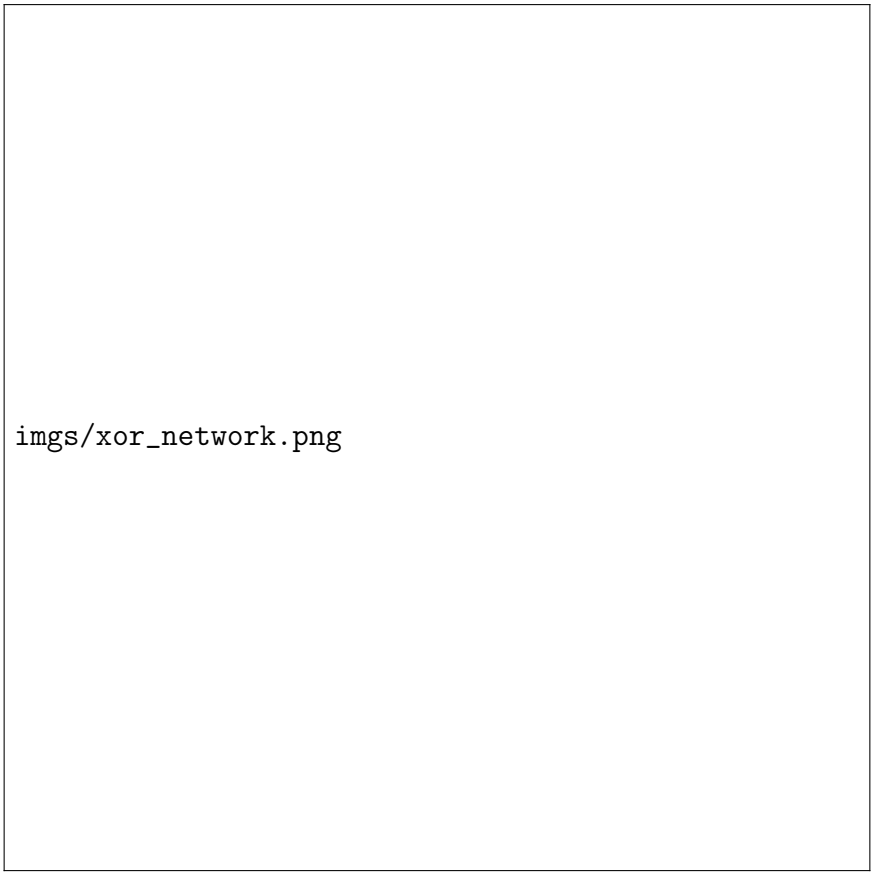



Figure 4: XOR solved by a 2-2-1 network: the hidden layer maps inputs to a linearly separable representation.

# 8 Gradient-Based Learning

## 8.1 Forward Propagation

- **Forward Pass**: Sequentially compute each layer's pre-activation and activation, passing results to the next layer. Store all intermediate values (needed for back-propagation).
- For layer $l$ with weight matrix $W^{(l)}$, bias $b^{(l)}$, and activation $\sigma$:

$$a^{(l)} = W^{(l)} h^{(l-1)} + b^{(l)}, \qquad h^{(l)} = \sigma\big(a^{(l)}\big)$$

$$\text{with } h^{(0)} = x \text{ (input)}, \quad \hat{y} = h^{(L)} \text{ (output of layer } L)$$

## 8.2 Backpropagation Algorithm

**Core Idea**: Use the **chain rule of calculus** to efficiently compute the gradient of the scalar loss $\mathcal{L}$ with respect to every weight and bias in the network, propagating gradients **from output to input**.

**Step 1 – Forward Pass:** Compute all $a^{(l)}$ and $h^{(l)}$ for $l = 1, \ldots, L$. Store them.

**Step 2 – Compute Loss:** At the output layer, compute scalar loss $\mathcal{L}(\hat{y}, y)$.

**Step 3 – Output Layer Error (delta):**

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} = (\hat{y} - y) \quad \text{[for MSE loss with linear output]}$$

$$\delta^{(L)} = \hat{y} - y \quad \text{[for cross-entropy loss with softmax output]}$$

**Step 4 – Propagate Error Backwards** (for $l = L-1, \ldots, 1$):

$$\delta^{(l)} = \left(W^{(l+1)}\right)^T \delta^{(l+1)} \odot \sigma'\left(a^{(l)}\right)$$

where $\odot$ denotes element-wise multiplication and $\sigma'$ is the derivative of the activation.

**Step 5 – Compute Gradients:**

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} \left(h^{(l-1)}\right)^T, \qquad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}$$

**Step 6 – Update Parameters** (gradient descent):

$$W^{(l)} \leftarrow W^{(l)} - \alpha \frac{\partial \mathcal{L}}{\partial W^{(l)}}, \qquad b^{(l)} \leftarrow b^{(l)} - \alpha \frac{\partial \mathcal{L}}{\partial b^{(l)}}$$

- **Chain Rule**: Backbone of backpropagation. If $z = f(y)$ and $y = g(x)$: $\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}$.
- **Efficiency**: Intermediate values (activations and deltas) are computed once and reused. Time complexity: $\mathcal{O}(L \times n^2)$ per sample where $n$ is width.
- **Memory**: Requires storing all forward-pass activations. Memory scales with depth and batch size (a known limitation for very deep networks).

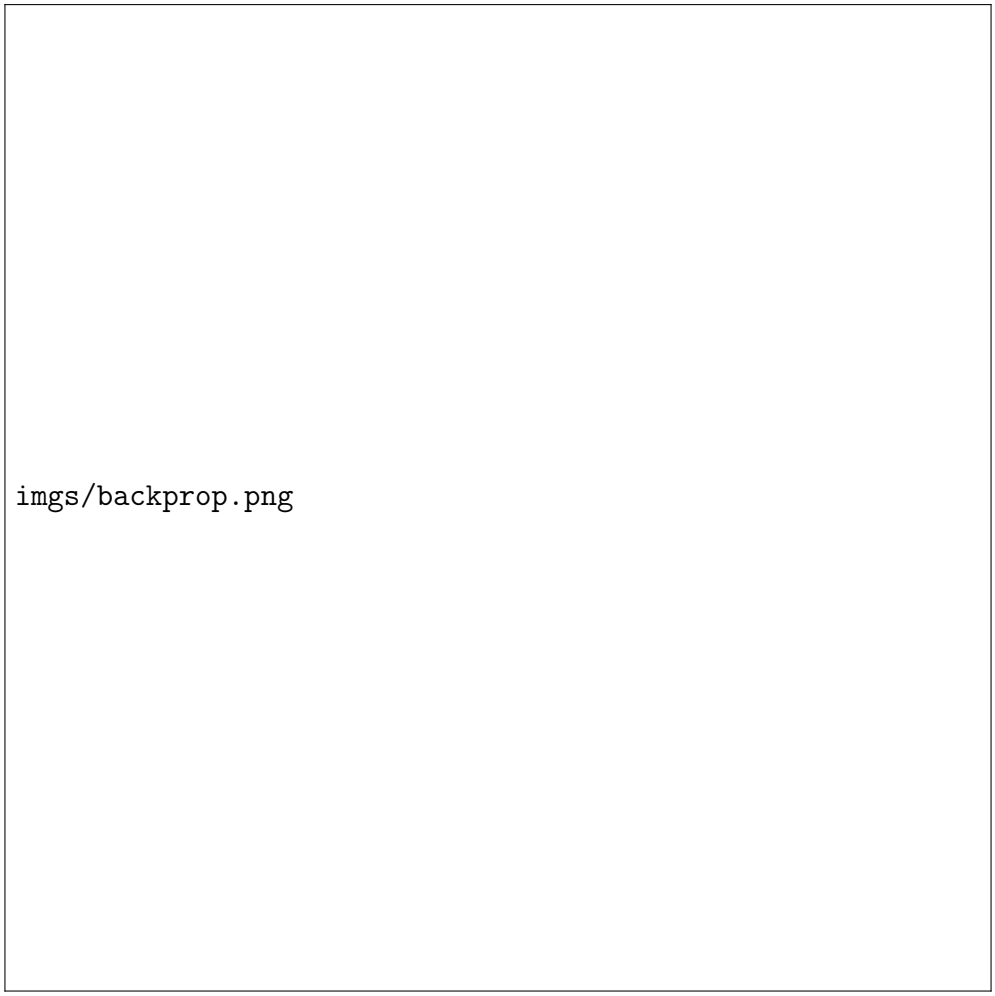



Figure 5: Backpropagation: forward pass (left to right, computing activations), backward pass (right to left, computing gradients via chain rule).

# 9 Hidden Units and Activation Functions

- **Sigmoid** $\sigma(x) = \dfrac{1}{1+e^{-x}}$: Output range $(0, 1)$. Used in output layer for **binary classification**. Suffers from **vanishing gradient** (saturates at 0 and 1) and outputs are not zero-centered. Rarely used in hidden layers.
- **Tanh** $\tanh(x) = \dfrac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$: Output range $(-1, 1)$. **Zero-centered** (preferred over sigmoid for hidden layers). Still suffers from vanishing gradient in deep networks.
- **ReLU** (Rectified Linear Unit) $f(x) = \max(0, x)$: Output range $[0, \infty)$. **Default activation function** for hidden layers. Fast to compute, does not saturate for $x > 0$ (no vanishing gradient in positive region), promotes sparse activations.

  - **Dying ReLU Problem**: If a neuron's input is always negative, its gradient is always 0 – it never updates ("dead neuron"). Common with large learning rates.

- **Leaky ReLU** $f(x) = \max(\alpha x, x)$, $\alpha = 0.01$: Fixes dying ReLU by allowing a small negative slope. **PReLU** (Parametric ReLU) learns $\alpha$ during training.

- **ELU** (Exponential Linear Unit):

$$f(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases}$$

  Smooth, mean activations closer to zero, handles dying ReLU. Computationally more expensive.

- **Softmax** $\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$: Output range $(0, 1)$ with outputs summing to 1. Used **only in the output layer** for multi-class classification. Converts logits to a probability distribution.

| Function | Formula | Range | Advantage | Disadvantage |
|---|---|---|---|---|
| **Sigmoid** | $1/(1+e^{-x})$ | $(0,1)$ | Probabilistic output | Vanishing gradient, not zero-centred |
| **Tanh** | $(e^x - e^{-x})/(e^x + e^{-x})$ | $(-1,1)$ | Zero-centred | Still vanishing gradient |
| **ReLU** | $\max(0,x)$ | $[0,\infty)$ | Fast, sparse | Dying ReLU |
| **Leaky ReLU** | $\max(0.01x, x)$ | $(-\infty,\infty)$ | Fixes dead ReLU | Fixed small slope |
| **ELU** | $x$ or $\alpha(e^x-1)$ | $(-\alpha,\infty)$ | Smooth, zero-mean | Slower compute |
| **Softmax** | $e^{z_i}/\sum e^{z_j}$ | $(0,1)^k$ | Multi-class probs | Output layer only |

## 9.1 Architecture Design

- **Input Layer**: Size equals the dimensionality of the input features (e.g., 784 for 28x28 MNIST images, 3072 for 32x32x3 CIFAR-10 images).

- **Output Layer**: Size and activation determined by the task. Binary classification: 1 sigmoid unit. Multi-class: $K$ softmax units. Regression: 1 linear unit.

- **Hidden Layers**: Number and size are hyperparameters. A common starting point is 1–3 hidden layers with 64–1024 units each. More layers = deeper representation.
- **Depth vs. Width**: Deeper networks can represent exponentially more functions. Wider layers increase capacity within a level. In practice, depth is preferred for structured data (images, text).
- **Skip / Residual Connections** (He et al., 2016): Connections that skip one or more layers. Alleviate vanishing gradient, allow training of very deep networks (ResNet-152). Output: $h^{(l+2)} = \sigma(h^{(l+2)}_{\text{plain}} + h^{(l)})$.

# Unit 3: Regularization and Optimization for Training Deep Models

## 10 Parameter Norm Penalties

- **Overfitting**: A model that memorizes training data (including noise) instead of learning the underlying pattern. High training accuracy, low test accuracy. The gap between train and test error is called **generalization gap**.
- **Regularization**: Adds a **penalty term** $\Omega(\theta)$ to the loss function to discourage overly complex models:

$$\tilde{\mathcal{L}}(\theta) = \mathcal{L}(\theta) + \lambda\,\Omega(\theta)$$

where $\lambda \geq 0$ is the **regularization strength** hyperparameter. Larger $\lambda$ = stronger penalty.

**L2 Regularization (Ridge / Weight Decay):**

- Penalty: $\Omega(\theta) = \frac{1}{2}\|W\|_2^2 = \frac{1}{2}\sum_i w_i^2$
- **Regularized Loss**: $\tilde{\mathcal{L}} = \mathcal{L} + \frac{\lambda}{2}\sum_i w_i^2$
- **Update Rule**: Gradient includes extra $\lambda w$ term:

$$w \leftarrow w - \alpha\left(\frac{\partial \mathcal{L}}{\partial w} + \lambda w\right) = w(1-\alpha\lambda) - \alpha\frac{\partial \mathcal{L}}{\partial w}$$

- Factor $(1-\alpha\lambda) < 1$ **shrinks** weights each step – hence "weight decay."
- **Geometry**: Penalizes large weights; encourages weights distributed across many features (no sparsity).
- **Bayesian Interpretation**: Equivalent to placing a **Gaussian prior** $P(w) \propto e^{-\lambda w^2/2}$ on weights.

**L1 Regularization (Lasso):**

- Penalty: $\Omega(\theta) = \|W\|_1 = \sum_i |w_i|$
- **Regularized Loss**: $\tilde{\mathcal{L}} = \mathcal{L} + \lambda\sum_i |w_i|$

- **Update Rule**: Gradient includes $\lambda \cdot \text{sign}(w)$:

$$w \leftarrow w - \alpha\left(\frac{\partial \mathcal{L}}{\partial w} + \lambda \cdot \text{sign}(w)\right)$$

- Produces **sparse** weight vectors – many weights become **exactly zero**. Acts as automatic **feature selection**.
- **Bayesian Interpretation**: Equivalent to placing a **Laplacian prior** $P(w) \propto e^{-\lambda|w|}$ on weights.

| Aspect | L2 (Ridge) | L1 (Lasso) |
|---|---|---|
| **Penalty term** | $\sum w_i^2$ | $\sum \lvert w_i \rvert$ |
| **Effect on weights** | Shrinks all weights proportionally | Sets some weights exactly to zero |
| **Sparsity** | No | Yes (feature selection) |
| **Solution uniqueness** | Always unique (strictly convex) | May not be unique |
| **Bayesian prior** | Gaussian | Laplacian |
| **Preferred when** | All features contribute | Many features are irrelevant |

## 10.1 Norm Penalties as Constrained Optimization

- L2 regularization is equivalent to constrained optimization: $\min_\theta \mathcal{L}(\theta)$ subject to $\|W\|_2^2 \leq r^2$.
- **KKT conditions** connect the Lagrange multiplier $\lambda$ to the constraint. Different $\lambda$ values correspond to different constraint radius $r$.
- **Geometric Interpretation**: The unconstrained optimum is pulled towards the origin (L2: elliptical constraint region; L1: diamond-shaped region, promoting corner solutions = sparsity).

## 10.2 Dataset Augmentation and Noise Robustness

- **Dataset Augmentation**: Artificially enlarges the training set by applying **label-preserving transformations**. Reduces overfitting without collecting new data.
  - Vision: horizontal/vertical flipping, random cropping, rotation, color jitter, scaling, cutout.

  - Audio: time stretching, pitch shifting, adding background noise.
  - NLP: synonym replacement, back-translation, random deletion.

- **Noise Robustness**: Adding **Gaussian noise** $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$ to inputs during training encourages the model to learn features that are stable under small perturbations. Equivalent to Tikhonov regularization for some models.

- **Label Smoothing**: Instead of hard one-hot labels, use soft labels: $\tilde{y}_k = (1-\epsilon)y_k + \epsilon/K$. Prevents overconfident predictions.

# 11 Semi-Supervised Learning, Multi-Task Learning, Early Stopping, Dropout, and Ensemble Methods

## 11.1 Semi-Supervised Learning

- **Semi-Supervised Learning**: Uses a **large amount of unlabeled data** alongside a small amount of labeled data. Unlabeled data helps learn the underlying data distribution.

- Common approach: train an autoencoder or language model on unlabeled data (unsupervised pre-training), then fine-tune on labeled data (supervised).

- Motivating scenario: Labeling data is expensive (medical imaging, legal documents), but unlabeled data is abundant.

## 11.2 Multi-Task Learning (MTL)

- **Multi-Task Learning**: Training a single model simultaneously on **multiple related tasks**, sharing representations across tasks. The tasks act as mutual regularizers.

- **Example**: A model trained on sentiment analysis and named entity recognition simultaneously can learn better text representations than models trained on either alone.

- Shared early layers + task-specific later layers. Inductive bias: features useful for multiple tasks are likely to generalize well.

## 11.3 Early Stopping

- **Concept**: Monitor **validation loss** during training. Stop training when validation loss stops improving and starts increasing, even if training loss continues to decrease.
- **Implementation**: Keep a copy of the best model weights seen so far. If validation loss does not improve for $p$ consecutive epochs (**patience**), restore the best model and stop.
- **Regularization Effect**: Constrains the effective number of optimization steps, preventing parameters from moving far from their initialization. For quadratic objectives, early stopping is equivalent to **L2 regularization** with $\lambda = 1/(\alpha T)$ where $T$ = number of steps and $\alpha$ = learning rate.
- **Advantages**: Free to apply (no change to loss function or architecture), can be combined with any other regularizer, saves compute by not running unnecessary epochs.




Figure 6: Early Stopping: training loss (solid) decreases continuously; validation loss (dashed) reaches a minimum then increases. Training is stopped at the validation minimum.

## 11.4 Dropout

- **Dropout** (Srivastava et al., 2014): Randomly **deactivates** (sets to zero) each neuron independently with probability $p$ (dropout rate) during each forward/backward training pass. A different random subset is dropped each mini-batch.
- **Training Phase – Inverted Dropout**: Each neuron is kept with probability $(1-p)$. Its output is scaled by $\frac{1}{1-p}$ to preserve expected activation magnitude.

$$\text{mask} \sim \text{Bernoulli}(1-p), \qquad \tilde{h} = \frac{h \odot \text{mask}}{1-p}$$

- **Test Phase**: All neurons are active (mask = 1). No scaling needed because

inverted dropout already handled it at training time. Full network produces a **geometric ensemble** prediction.

- **Typical Rates**: $p = 0.5$ for fully-connected hidden layers; $p = 0.1$–$0.2$ for input layer. Not typically used on output layer.
- **Why it Works – Three Complementary Explanations**:
  1. **Prevents co-adaptation**: Neurons cannot rely on specific other neurons being present, so each must learn useful features independently.
  2. **Ensemble interpretation**: Dropout implicitly trains $2^n$ different sub-networks (one per mask combination) and averages their predictions at test time – like **bagging** at negligible extra cost.
  3. **Adds noise**: The random mask injects multiplicative noise, acting as a regularizer similar to noise injection but more powerful.
- **Limitation**: Increases training time (network needs more iterations to converge). Not as effective in CNNs as in FC layers; **SpatialDropout** (drops entire feature maps) is preferred for CNNs.

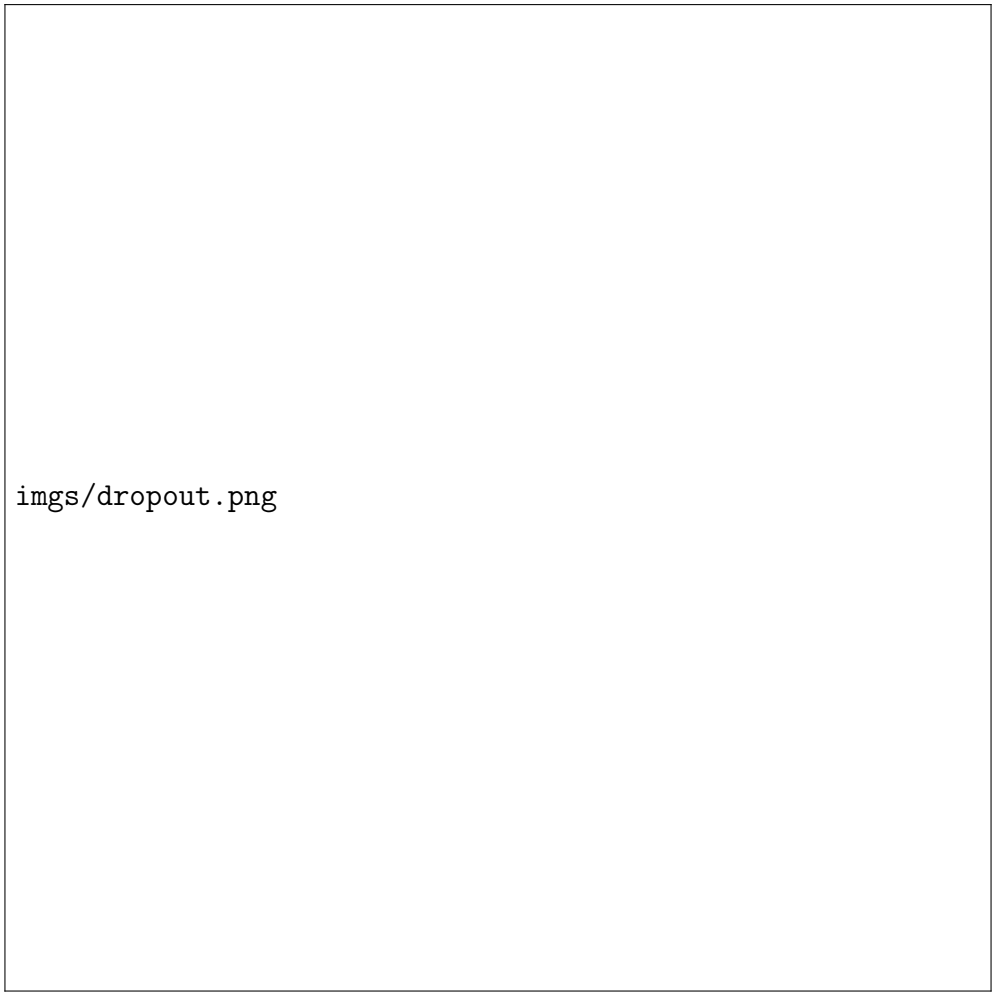



Figure 7: Left: Standard neural network. Right: After applying 50% dropout – crossed neurons are deactivated for this training step. A different mask is used each iteration.

## 11.5 Bagging and Ensemble Methods

- **Bagging** (Bootstrap Aggregating): Train $k$ models on $k$ **bootstrapped** (sampled with replacement) datasets. Average predictions (regression) or majority vote (classification). Reduces **variance** without increasing bias.
- **Random Forests**: Bagging of decision trees + random feature selection at each split. Very effective in practice.
- **Boosting**: Sequentially train models, each focusing on samples misclassified by previous models. Reduces **bias** and variance. Examples: AdaBoost, Gradient Boosting, XGBoost.
- **Stacking**: Train a meta-learner (second-level model) on the output of multiple base learners. Often outperforms individual models.

### 11.6 Adversarial Training

- **Adversarial Examples**: Inputs with small, imperceptible perturbations crafted to fool a model. Example: an image of a panda + tiny noise = classified as "gibbon" with high confidence.
- **Adversarial Training**: Add adversarial examples to the training set. The model is trained on both clean and perturbed inputs, improving **robustness**.
- **FGSM** (Fast Gradient Sign Method): Most common attack/augmentation method.

$$\tilde{x} = x + \epsilon \cdot \text{sign}(\nabla_x \mathcal{L}(\theta, x, y))$$

## 12 Challenges in Neural Network Optimization

- **Vanishing Gradient**: In deep networks, gradients are multiplied by the weight matrix and activation derivative at each layer during backprop. Sigmoid/Tanh activations squash inputs into $[0, 1]$ or $[-1, 1]$ with derivatives $< 1$. Over many layers, gradients decay **exponentially to zero**.

  $$\frac{\partial \mathcal{L}}{\partial h^{(1)}} \propto \prod_{l=2}^{L} \left(W^{(l)} \cdot \sigma'\left(a^{(l)}\right)\right) \to 0 \text{ as } L \to \infty$$

  **Solutions**: ReLU activations, Batch Normalization, Residual Connections (skip connections).

- **Exploding Gradient**: If $\|W\| > 1$ or activations are large, gradients grow **exponentially**, causing numerical overflow and extremely large parameter updates. Especially common in RNNs.

  $$\text{Gradient Clipping: } g \leftarrow g \cdot \frac{\text{threshold}}{\|g\|} \text{ if } \|g\| > \text{threshold}$$

- **Saddle Points**: Points where the gradient is zero but which are neither a local minimum nor maximum (some directions curve up, others down). **Very common in high-dimensional loss surfaces**. SGD's inherent noise helps escape saddle points; second-order methods can detect them.
- **Ill-Conditioning of the Hessian**: The Hessian matrix $H$ of the loss has a very large **condition number** $\kappa(H) = \lambda_{\max}/\lambda_{\min}$. This causes *ravines* in the loss surface – small steps overshoot in some directions, slow progress in others.

Adaptive LR methods (Adam) address this.

- **Flat Regions (Plateaus)**: Regions where the gradient is near zero everywhere. Training stalls. SGD momentum helps traverse plateaus.
- **Poor Local Minima**: Although less common than once thought, poor local minima can trap optimization. In high dimensions, most local minima have similar loss values to the global minimum.

# 13 Basic Optimization Algorithms

## 13.1 Momentum

- **Concept**: Accumulates an **exponentially decaying moving average** of past gradients (velocity $v$). Helps accelerate in consistent gradient directions and dampens oscillations in ravines.

$$v \leftarrow \beta v - \alpha g, \qquad \theta \leftarrow \theta + v$$

$$\text{Typical: } \beta = 0.9, \quad \alpha = \text{learning rate}, \quad g = \nabla_\theta \mathcal{L}$$

- **Nesterov Momentum**: Computes gradient *after* applying the velocity step ("look-ahead"), leading to faster and more stable convergence.

$$\theta_{\text{ahead}} = \theta + \beta v, \qquad v \leftarrow \beta v - \alpha \nabla_\theta \mathcal{L}(\theta_{\text{ahead}})$$

# 14 Algorithms with Adaptive Learning Rates

**AdaGrad (Adaptive Gradient):**

- Accumulates the sum of **all past squared gradients**. Parameters with historically large gradients receive smaller effective learning rates.

$$r \leftarrow r + g^2, \qquad \theta \leftarrow \theta - \frac{\alpha}{\sqrt{r + \epsilon}} \odot g$$

- **Limitation**: $r$ grows monotonically; learning rate decays to near zero over time, halting learning. Best for **sparse data / NLP**.

**RMSProp (Root Mean Square Propagation):**

- Fixes AdaGrad's monotonic decrease using an **exponentially decaying average** of squared gradients. Each parameter gets its own adaptive LR.

$$r \leftarrow \rho\, r + (1-\rho)\, g^2, \qquad \theta \leftarrow \theta - \frac{\alpha}{\sqrt{r+\epsilon}} \odot g$$

Typical: $\rho = 0.9,\ \epsilon = 10^{-8}$

- **Best for**: Non-stationary objectives, **RNNs**, online learning.

**Adam (Adaptive Moment Estimation)** – Kingma & Ba, 2014:

- Combines **Momentum** (first moment of gradient) and **RMSProp** (second moment of squared gradient). Most widely used optimizer in deep learning.

$$m \leftarrow \beta_1 m + (1-\beta_1)\, g \quad \text{[first moment – momentum]}$$

$$v \leftarrow \beta_2 v + (1-\beta_2)\, g^2 \quad \text{[second moment – RMSProp]}$$

$$\hat{m} = \frac{m}{1-\beta_1^t}, \qquad \hat{v} = \frac{v}{1-\beta_2^t} \quad \text{[bias correction]}$$

$$\theta \leftarrow \theta - \frac{\alpha\, \hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

Defaults: $\beta_1 = 0.9,\ \beta_2 = 0.999,\ \epsilon = 10^{-8},\ \alpha = 0.001$

- **Bias Correction**: At early steps, $m$ and $v$ are biased towards zero (initialized as 0). Dividing by $(1-\beta^t)$ corrects this, especially important in the first few iterations.
- **Robustness**: Works well across a wide range of architectures and tasks with minimal hyperparameter tuning.

| Optimizer | Key Idea | Best For | Hyperparams |
|---|---|---|---|
| **SGD** | Fixed LR gradient step | Simple convex problems | $\alpha$ |
| **Momentum** | Accumulate velocity | Smooth loss, ravines | $\alpha, \beta$ |
| **Nesterov** | Look-ahead momentum | Same as Momentum | $\alpha, \beta$ |
| **AdaGrad** | Sum of past $g^2$ | Sparse data, NLP | $\alpha$ |
| **RMSProp** | Decaying avg of $g^2$ | RNNs, non-stationary | $\alpha, \rho$ |
| **Adam** | Momentum + RMSProp | General deep learning | $\alpha, \beta_1, \beta_2$ |

# 15 Parameter Initialization Strategies

- **Symmetry Breaking**: If all weights are zero, all neurons compute the same gradient and learn identically – the network fails to differentiate. Non-zero random initialization **breaks symmetry**.
- **Scale Problem**: If weights are too large, activations and gradients **explode**. If too small, they **vanish**. Proper initialization keeps variance approximately constant across layers.
- **Zero Initialization**: Never initialize all weights to zero (symmetry problem). Biases can be initialized to zero.
- **Random Gaussian/Uniform**: Basic approach. Risk of vanishing/exploding depending on scale.
- **Xavier / Glorot Initialization** (Glorot & Bengio, 2010): Designed for **sigmoid / tanh** activations. Maintains variance of activations and gradients across layers.

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}},\ +\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}\right) \quad \text{or} \quad W \sim \mathcal{N}\left(0,\ \frac{2}{n_{\text{in}}+n_{\text{out}}}\right)$$

- **He / Kaiming Initialization** (He et al., 2015): Designed for **ReLU** activations. Since ReLU zeros out half its inputs on average, variance needs to be doubled compared to Xavier.

$$W \sim \mathcal{N}\left(0,\ \frac{2}{n_{\text{in}}}\right)$$

- **Key Rule**: Use Xavier for sigmoid/tanh; use He for ReLU/Leaky ReLU.

# 16 Approximate Second-Order Methods and Batch Normalization

## 16.1 Batch Normalization

- **Batch Normalization** (Ioffe & Szegedy, 2015): Normalizes the pre-activation of each layer to have **zero mean and unit variance** across each mini-batch. Applied before or after the activation function.

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y_i = \gamma \hat{x}_i + \beta$$

$$\mu_B = \frac{1}{B}\sum_{i=1}^{B} x_i, \qquad \sigma_B^2 = \frac{1}{B}\sum_{i=1}^{B}(x_i - \mu_B)^2$$

- $\gamma$ (scale) and $\beta$ (shift) are **learnable parameters**, allowing the network to undo normalization if needed.
- **At Test Time**: Uses running mean and variance accumulated during training (not batch statistics).
- **Benefits**:
  1. **Mitigates vanishing/exploding gradient** by keeping activations in a healthy range.
  2. **Allows higher learning rates** – training is much faster.
  3. **Acts as regularizer** – reduces need for Dropout in some architectures.
  4. Makes training less sensitive to weight initialization.